

Log File Anomaly Explainer

Project Documentation - AI-Powered Log Analysis, Root Cause Explanation and Markdown Report Generation

Item	Details
Project Type	Python CLI + Streamlit web application
Core Capability	Detects error blocks from .log files and explains the root cause using a local Ollama LLM
Storage	SQLite database for analysis history and generated report records
Output	Developer-friendly Markdown incident report
Target Users	Developers, DevOps teams, support engineers and recruiters evaluating practical AI software skills

1. Abstract

Log File Anomaly Explainer is an AI-powered Python application designed to reduce the time required to understand failures inside large log files. The system accepts a .log file, detects error or critical blocks using pattern-based parsing, extracts useful surrounding context, sends the meaningful error context to a local Ollama language model, and produces a clear Markdown report containing the probable root cause and suggested fixes. The project combines practical software engineering, local LLM integration, a Streamlit user interface, SQLite-based persistence, and command-line usability. It is built as a lightweight proof of concept that demonstrates how AI can support incident analysis while keeping sensitive logs on the user machine.

2. Problem Statement

Modern applications generate thousands of log lines during normal execution. When an issue occurs, developers and support teams often spend significant time searching for the exact failure line, reading stack traces, understanding what happened before and after the error, and preparing a short incident summary. This manual process is slow, repetitive and error-prone, especially for fresh developers or support engineers handling unfamiliar systems.

- Large log files make it difficult to quickly identify the actual error block.
- Stack traces and exception messages are often technical and hard to convert into action items.
- Manual root cause analysis takes time during production incidents and support tickets.
- Sharing findings requires clean documentation, but report writing is usually done separately.

3. Proposed Solution

The proposed solution is a local AI-assisted log analysis tool. Instead of uploading logs to a third-party service, the application reads the file locally, detects high-signal error patterns such as ERROR, CRITICAL, FATAL, Exception and Traceback, captures surrounding context, and uses Ollama as a local LLM runtime to generate a developer-readable explanation. The final output is saved as a Markdown report that can be shared with a team or attached to a support ticket.

- Automated parsing reduces manual scanning effort.
- Context extraction improves the quality of the AI explanation.
- Ollama keeps analysis local and avoids external API key dependency.
- Streamlit provides a simple UI for demonstration and recruiter review.
- SQLite stores analysis history, making the project more complete than a basic script.

4. Objectives

- Build a functional CLI and Streamlit interface for log file analysis.
- Identify meaningful error blocks and related context from raw log files.

- Use a local LLM through Ollama to explain the issue in simple technical language.
- Generate a structured Markdown report with root cause, impact and suggested fixes.
- Store useful analysis records in SQLite for traceability and future review.
- Keep the stack minimal, readable and suitable for a rapid proof of concept.

5. Recruiter-Focused Highlights

Evaluation Area	What This Project Demonstrates
Python Engineering	File handling, modular code structure, regex parsing, CLI execution and exception-safe flow.
AI Integration	External local LLM integration using Ollama for root cause explanation and fix suggestions.
Full-Stack Thinking	Streamlit UI, backend modules, SQLite persistence and report generation workflow.
DevOps Relevance	Direct use case for logs, incidents, debugging, support automation and production troubleshooting.
Documentation	Readable README, Markdown reports and structured project documentation for handoff.
Privacy Awareness	Local analysis avoids sending sensitive application logs to cloud APIs.

6. Tech Stack

Layer	Technology	Purpose
Programming Language	Python	Core development language for CLI, parsing, backend logic and automation.
Frontend	Streamlit	Interactive web interface for uploading log files and viewing analysis output.
Database	SQLite	Stores analysis metadata, history and generated report details locally.
AI / LLM Runtime	Ollama	Runs local models such as Llama or Mistral without API keys.
Parsing	Python re module	Detects error keywords, traceback patterns, timestamps and severity.
Report Format	Markdown	Produces clean, readable and GitHub-friendly incident reports.
Source Control	Git + GitHub	Version control, recruiter visibility and project collaboration.

7. System Workflow

1. User uploads or provides a .log file through the Streamlit app or CLI.
2. The parser reads the file and scans for error indicators such as ERROR, CRITICAL, FATAL, Exception or Traceback.
3. The system extracts the primary error block with before-and-after context lines.
4. Severity, timestamp and important trace information are normalized into a structured dictionary.
5. The extracted context is sent to the Ollama LLM explainer module.
6. The LLM returns a concise explanation, probable cause and suggested technical fixes.
7. The report generator formats the result into a Markdown incident report.
8. The application saves the report and stores analysis metadata in SQLite.

8. Architecture

The architecture follows a simple modular pipeline. Each component has one responsibility, making the system easy to test, extend and explain during interviews. The graph diagram below replaces the earlier one-line architecture text and gives a clearer visual view of the data flow.

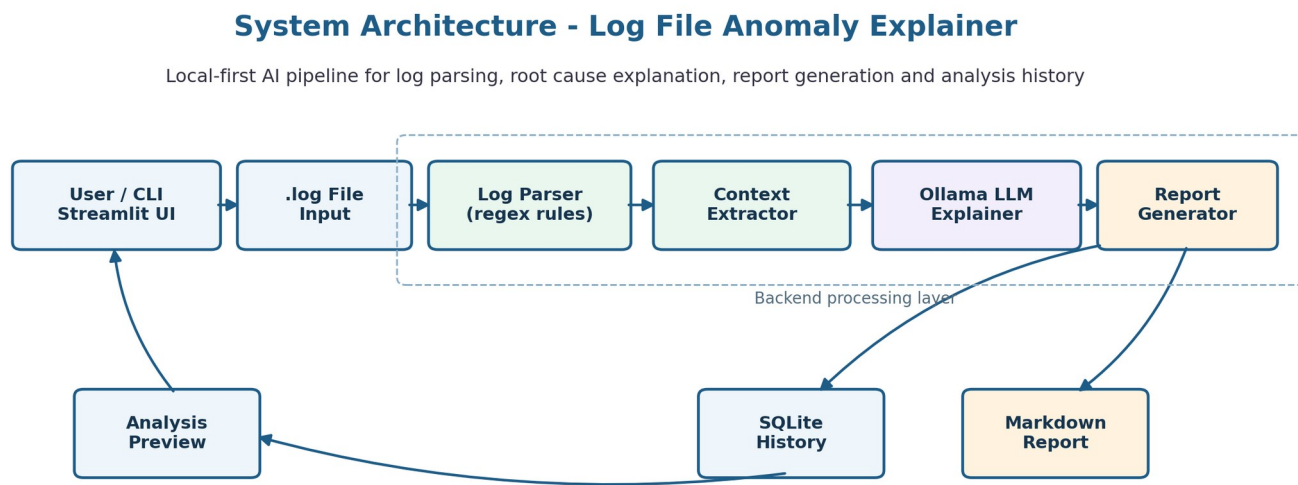


Figure 1: Graph-based architecture diagram showing the local AI log-analysis pipeline.

Component	Responsibility
Streamlit App	Handles file upload, user actions, report preview and analysis history display.
CLI Entry Point	Allows direct terminal-based analysis for developer workflows.
Log Parser	Finds error blocks, extracts context and identifies severity.
Context Extractor	Captures useful before-and-after log lines for better LLM accuracy.
LLM Explainer	Sends useful error context to Ollama and receives explanation/fix suggestions.
Report Generator	Creates a structured Markdown report from parser and LLM outputs.
SQLite Database	Persists metadata such as filename, timestamp, severity and report path.

9. Module Description

Module/File	Description
app.py	Main Streamlit frontend. It connects UI actions with backend parsing, explanation and report generation.
backend/log_parser.py	Regex-based log scanner that detects error blocks, severity and surrounding context.
backend/llm_explainer.py	Ollama integration layer that converts raw error context into human-readable root cause analysis.
backend/report_generator.py	Formats final findings into a Markdown report.
backend/main.py	CLI entry point for terminal-based usage.
frontend/database.py	SQLite helper functions for storing and retrieving analysis history.
frontend/styles.py and utils.py	UI support files for styling and reusable frontend utilities.
reports/ and uploads/	Runtime folders for generated reports and uploaded logs.

10. Key Features

- Upload and analyze .log files through a clean Streamlit interface.
- Detect common error signals such as ERROR, CRITICAL, FATAL, Exception, Traceback and stack traces.
- Extract useful before-and-after context around the failure line.
- Identify severity levels such as ERROR, CRITICAL and UNKNOWN.
- Generate local LLM explanations without relying on paid external APIs.
- Produce Markdown reports suitable for GitHub, support tickets or incident documentation.
- Maintain SQLite analysis history for better traceability.
- Use a lightweight, recruiter-friendly and easy-to-run Python stack.

11. Database Usage

SQLite is used as a simple embedded database because it requires no server setup and is ideal for proof-of-concept applications. In this project, SQLite helps demonstrate persistence by storing previous analysis results, report paths and timestamps. This makes the application more realistic than a single-run script because users can review earlier log analyses from the interface.

Possible Field	Purpose
id	Unique record identifier.
filename	Name of the analyzed log file.
severity	Detected severity such as ERROR or CRITICAL.
timestamp	Analysis time or log event time.
summary	Short AI-generated explanation of the incident.
report_path	Location of the generated Markdown report.

12. Expected Input and Output

Item	Description
Input	Plain-text .log file from an application, server, service or sample dataset.
Processing	Regex-based detection, context extraction and Ollama-based explanation.
Output	Markdown report containing detected issue, context summary, root cause and suggested fixes.
User Benefit	Faster debugging, clearer incident communication and reduced manual log scanning.

13. Sample Report Sections

- Detected issue and severity
- Important log context
- AI-generated root cause explanation
- Suggested fix steps
- Timestamp and analyzed file information
- Final incident summary suitable for team sharing

14. Limitations

- The current parser focuses on common error keywords and may need custom rules for specialized log formats.
- The quality of explanations depends on the selected Ollama model and the clarity of the extracted context.
- Very large production logs may require streaming or chunk-based parsing for better memory efficiency.
- The tool supports analysis assistance; final production decisions should still be reviewed by an engineer.

15. Future Enhancements

- Support JSON logs, Syslog, Apache/Nginx logs and Kubernetes pod logs.
- Add real-time log monitoring mode for continuous anomaly detection.
- Enable multiple file comparison to identify repeated incidents.
- Add dashboard charts for error frequency, severity trend and common root causes.
- Add Docker support for easy deployment and reproducible setup.
- Add unit tests and CI workflow for better engineering maturity.

16. Conclusion

Log File Anomaly Explainer is a practical AI software project that connects real developer pain points with a simple, useful solution. It demonstrates Python programming, log parsing, LLM integration, UI development, local persistence and documentation skills in one complete workflow. For recruiters, the project shows applied problem solving, awareness of DevOps workflows, and the ability to convert AI capability into a usable engineering tool.